CODEALPHA / TASK 3

Handwritten Character Recognition

Project Development Report

A chronological record of the EMNIST Letters dataset, CNN experiments, model errors, fixes, and final evaluation.

Project objective

Train a convolutional neural network to identify a single handwritten lowercase letter. I first established an EMNIST baseline, used error analysis to guide model changes, then tested whether the selected checkpoint could handle photographed handwriting after preprocessing.

OFFICIAL TEST ACCURACY	MACRO F1	BEST VALIDATION	PHOTO CHECK
95.64%	**95.65%**	**95.74%**	**26 / 26**
VGG-style CNN 20,800 EMNIST images	Official EMNIST test split	VGG checkpoint, epoch 30	After Method C + i/l rule; small sample

Development period: 6-18 October 2026

Scope: model and dataset work only. Deployment, interface design, and unrelated project details are excluded.

01 / INITIAL SETUP

Dataset and first CNN

I chose **EMNIST Letters** rather than MNIST because the task was handwritten alphabet recognition. The dataset provides one grayscale character per image, so the first model was designed as a single-character classifier.

Item	Configuration used
Dataset	EMNIST, split="letters"
Input	1-channel grayscale image, 28 x 28 pixels
Official training split	124,800 images
Official test split	20,800 images; 800 per letter
Target labels	26 lowercase letters a-z
Initial output layer	27 outputs: the EMNIST label convention includes an unused class 0 / N/A slot

The first CNN used four convolutional stages with 8, 16, 32 and 64 channels, batch normalisation, ReLU and max pooling, followed by a linear classifier. Training used Adam (learning rate 0.001) and cross-entropy loss.

The initial run reached **95.45% training accuracy and 93.14% test accuracy**. Later runs of the early architecture reached about 93.7-94.0% test accuracy, but the score began to plateau.

FIRST-RUN TRAIN

95.45%

Initial CNN

FIRST-RUN TEST

93.14%

Initial CNN

OBSERVED PLATEAU

~93.9%

Test score on later early runs

What I found

Training accuracy continued to rise while test accuracy and test loss changed very little. The train-test gap was around 2-2.5 percentage points: a generalisation gap, but not evidence of severe overfitting by itself.

What I did next

I added epoch-level loss/accuracy tracking, then generated a confusion matrix and per-class precision, recall and F1. This shifted the next changes from guesswork to specific character-level errors.

02 / ERROR ANALYSIS AND CONTROLLED TRIALS

What failed, and what changed

The early classification reports showed that the model was not equally strong on every letter. The recurring weak classes were **i, l, g and q**; the main error pairs were i ↔ l and g ↔ q. These lowercase forms often have similar strokes, so overall accuracy concealed a consistent per-class weakness.

One evaluation mistake also had to be fixed: the first grid of incorrect predictions showed mostly actual “a” examples because the code stopped after collecting the first few errors from a class-ordered test set. I changed it to scan the complete test dataset first, then sample mistakes across all classes.

I also corrected the EMNIST orientation handling. The transpose operation had to be applied consistently wherever images were trained, validated, or inferred. Otherwise, the model could receive an image layout that did not match the training input.

Experiment record

Experiment / change	Test result	Finding
Initial CNN	93.14%	Baseline; noticeable train-test gap.
Augmentation only	93.68%	Did not beat the prior baseline.
Orientation correction + augmentation	94.16%	Improved alignment and overall score.
Spatially richer CNN + augmentation	94.46%	Retaining a larger final feature map helped.
Milder augmentation	94.59%	Small improvement over stronger augmentation.
No augmentation control	93.29%	Worse generalisation than mild augmentation.
Fine-tuned earlier checkpoint	94.64%	A small gain; not consistent across letters.
Later v4 run	94.44%	No meaningful gain over the 94.46% run.

Important evaluation note. These intermediate scores came from repeated experiments against the same official test split. They were useful for debugging, but repeated test-guided choices mean they should not be treated as an untouched final benchmark.

Checkpoint issue

During the Colab workflow, a save attempt referred to a model variable that was not present in the active runtime. I verified which model object actually existed and saved each experiment’s state dictionary to a separate checkpoint in Google Drive instead of overwriting earlier versions.

03 / VALIDATION-FIRST TRAINING

A cleaner model-selection process

After seeing how much the test split had been used during tuning, I rebuilt the experiment around a reproducible **90/10 split of the official training data**. This provided 112,320 training images and 12,480 validation images. I trained a fresh CNN with batch normalisation, dropout and mild affine augmentation, saved the best validation checkpoint, and used early stopping.

The best checkpoint reached **94.94% validation accuracy** at epoch 5. The model still used 27 outputs, however, even though output class 0 was unused. The diagnostic run confirmed that the model never predicted class 0; the output mapping was simplified to 26 letters in the later VGG model.

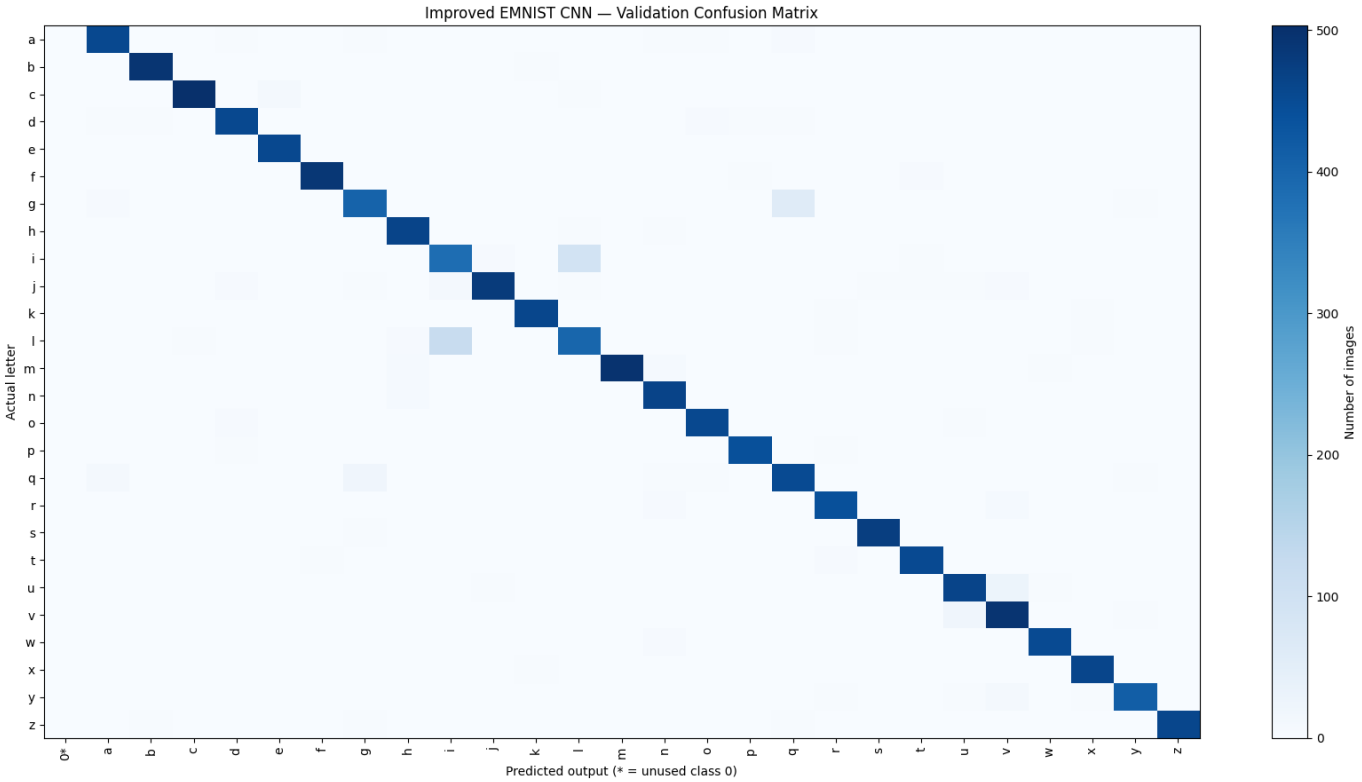


Figure 1. Validation confusion matrix for the 94.94% CNN checkpoint. Most predictions lie on the diagonal; the visible off-diagonal blocks include similar letter pairs.

The validation error analysis showed 632 mistakes out of 12,480. The two biggest problem groups were i ↔ l (214 combined errors) and g ↔ q (79); u ↔ v contributed another 47. Together, these three groups made up **340 of 632 errors (53.8%)**.

Lowest validation accuracy	Accuracy
l	74.53%
i	78.85%
g	85.59%
q	92.21%

This result made the next experiment clear: test an architecture with stronger feature extraction, rather than continuing to make small random changes to the original CNN.

04 / STRONGER ARCHITECTURE

VGG-style CNN: selected checkpoint

I benchmarked a deeper VGG-style CNN while keeping the dataset split and image orientation correction consistent. It uses repeated 3 x 3 convolution layers, batch normalisation, ReLU and max pooling, with a dropout-based classifier. The final classifier outputs **26 classes** for a-z.

Training used AdamW (learning rate 0.001, weight decay 0.0001), a learning-rate scheduler and early stopping. The best validation accuracy was **95.74% at epoch 30**; training accuracy at that point was 97.21%. The VGG checkpoint replaced the 94.94% CNN checkpoint as the best model.

OFFICIAL TEST ACCURACY	MACRO F1	WEIGHTED F1	TEST LOSS
95.64%	95.65%	95.65%	0.1403
19,893 / 20,800 correct	26 letters	26 letters	VGG-style checkpoint

Official EMNIST test evaluation

The selected checkpoint classified **19,893 of 20,800** official test images correctly and misclassified 907. The per-class classification report gave a macro F1 of 95.65% and a weighted F1 of 95.65%.

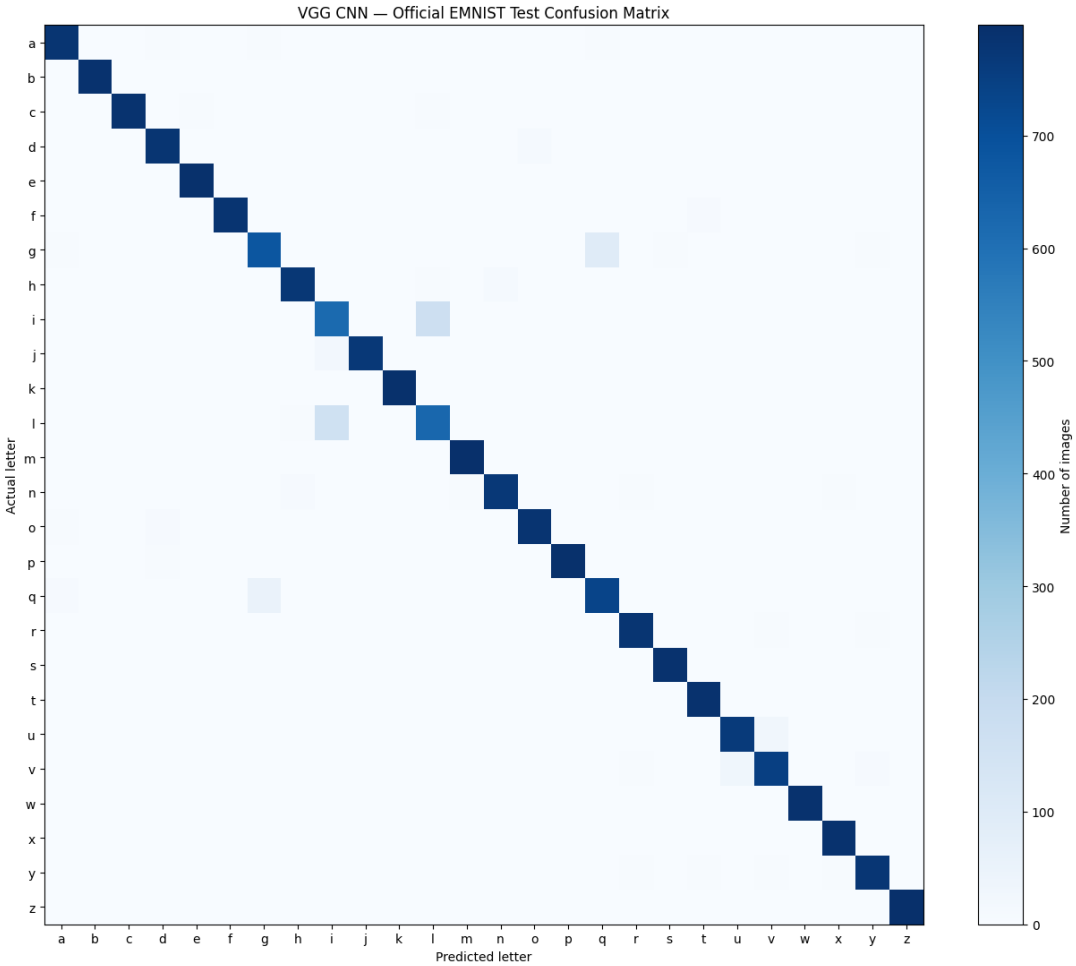


Figure 2. Confusion matrix for the VGG-style CNN on the official EMNIST test split.

05 / WHAT REMAINS DIFFICULT

The remaining errors are concentrated

The final test confusion matrix and the misclassified-image grid agree: most classes are recognised well, but the model repeatedly swaps visually similar letters. The largest pairs were:

Actual → predicted	Count
i → l	171
l → i	161
g → q	96
q → g	50
v → u	31
u → v	26

i ↔ **l** and **g** ↔ **q** account for 478 of the 907 test mistakes. Class-wise test accuracy was 77.25% for i, 78.38% for l, 85.25% for g and 91.88% for q. Their F1-scores were 0.7706, 0.7750, 0.8823 and 0.8958 respectively. The other letters were mostly above 93%.

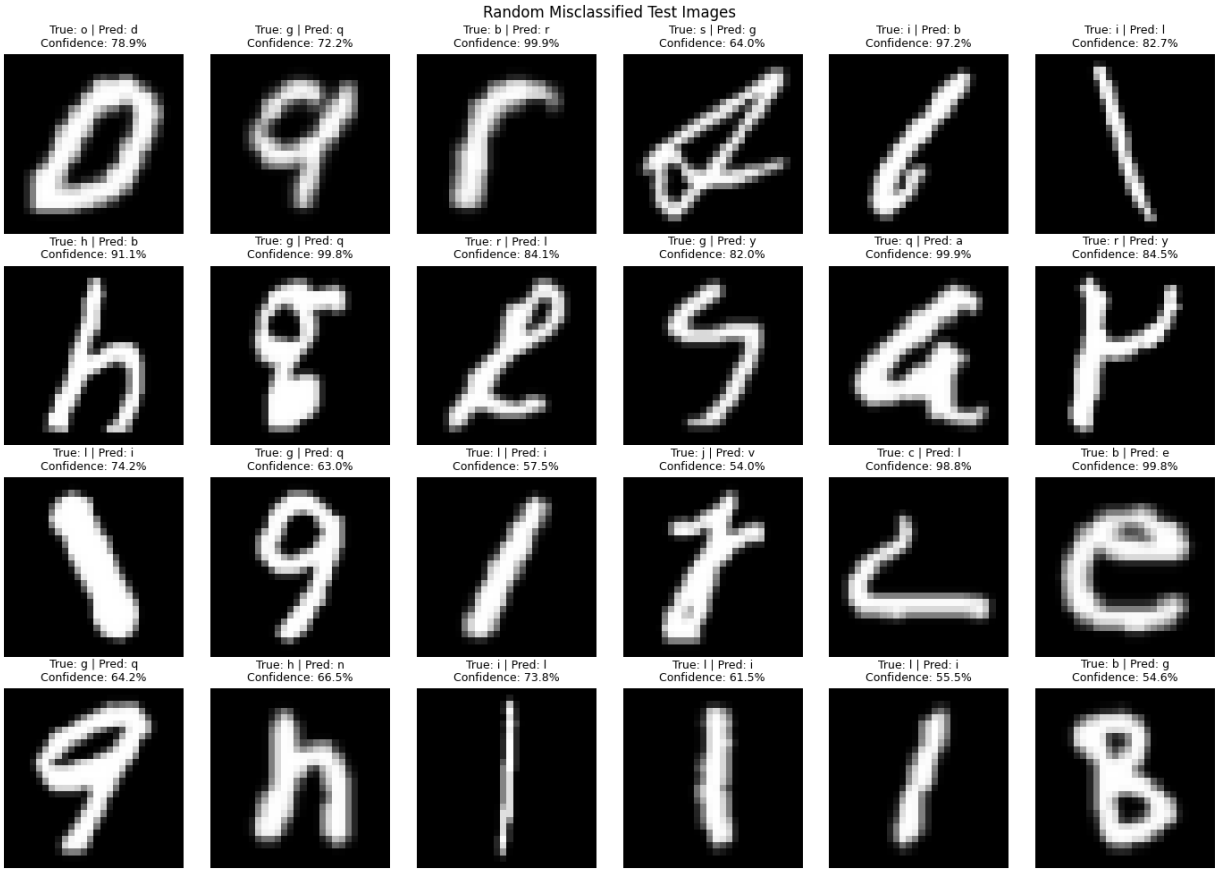


Figure 3. Examples of incorrect predictions from the VGG test evaluation. Several mistakes occur between shapes that share much of the same stroke structure.

This error pattern was not solved by confidence thresholds: the model sometimes assigned high confidence to a wrong character. The errors need to remain visible in the project report rather than being hidden by the overall accuracy.

06 / REAL-IMAGE INFERENCE

Preprocessing mattered more than another training run

The next problem appeared when I uploaded photographs of my own handwriting. These images did not look like EMNIST inputs: they contained paper background, uneven lighting, borders and strokes at different positions. Running the photograph through the original preprocessing produced only **4 correct predictions out of 26**. One upload attempt also used a binary EMNIST **.idx** label file instead of a character image; I corrected that by using actual PNG/JPG handwriting images.

I compared preprocessing methods while keeping the trained VGG checkpoint fixed. Method B improved cropping and contrast handling. Method C first detected the bright paper region and then extracted, normalised and resized the character to the model input. No model retraining was performed for these checks.

Input preprocessing	Correct	Accuracy
Method A: existing preprocessing	4 / 26	15.38%
Method B: improved paper/ink processing	22 / 26	84.62%
Method C: paper detection + ink extraction	25 / 26	96.15%
Method C + conservative i/l dot rule	26 / 26	100.00%

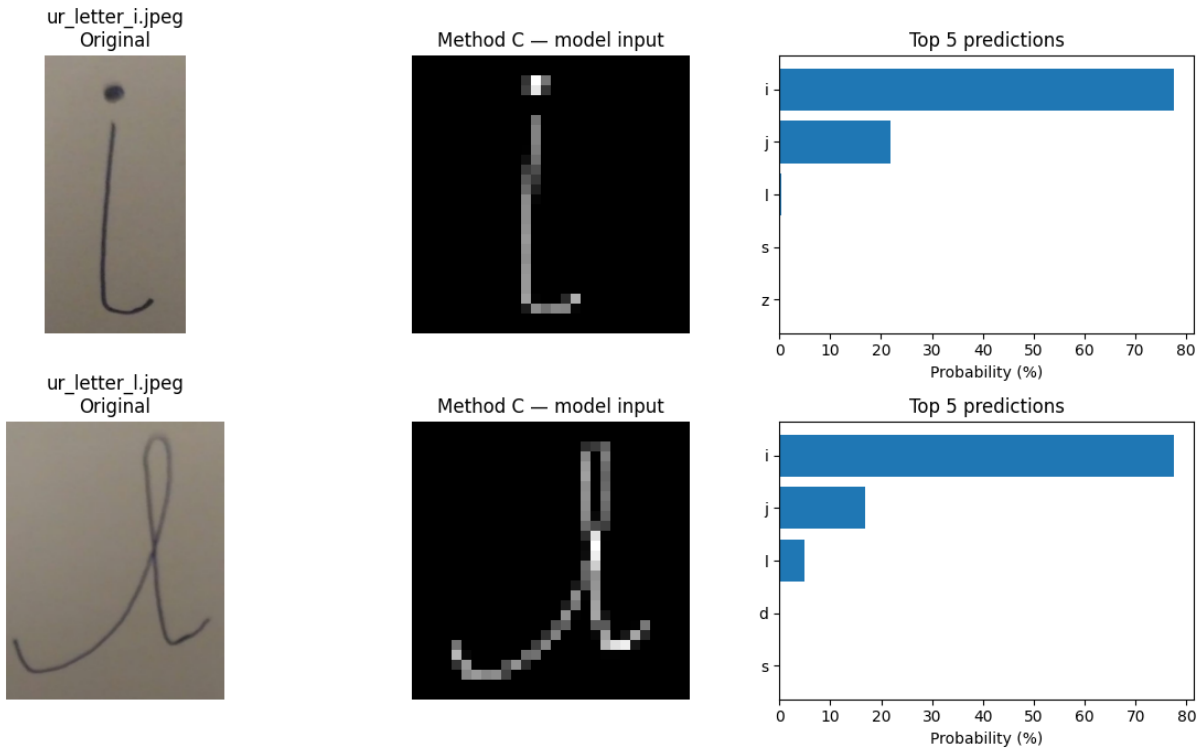


Figure 4. Original photographs and their Method C model inputs for handwritten i and l, alongside the model's top predictions. The l sample was still classified as i before the rule-based correction.

The final small-sample check reached 25/26 with Method C. A conservative rule then examined whether a detached dot was present when the CNN predicted i: dot present kept i; dot absent changed it to l. This corrected the remaining l sample and produced 26/26 on this set.

These photo results are not the official EMNIST test accuracy. They describe only 26 labelled photographs. The 100% figure reflects this specific sample after adding a hand-built rule; it does not establish 100% generalisation to new handwriting.

07 / CURRENT STATUS

Final result and remaining limits

The best recorded model is **emnist_vgg_benchmark_val_best.pth**. It achieved 95.74% validation accuracy and 95.64% on the 20,800-image EMNIST test split, with macro and weighted F1 scores of 95.65%. The real-image pipeline currently uses Method C preprocessing plus the detached-dot i/l correction.

What the experiments established

Finding	Evidence from this project
Architecture	The VGG-style CNN improved validation accuracy from 94.94% to 95.74%.
Data handling	Applying orientation correction consistently and mapping the final classifier to 26 letters resolved important input/label alignment issues.
Augmentation	Mild affine augmentation performed better than the no-augmentation control in the spatial CNN trials.
Error distribution	The main unresolved EMNIST confusions remain i ↔ l and g ↔ q.
Photographed handwriting	Method C preprocessing raised the 26-image sample score to 25/26; the i/l rule corrected the last error.

Limitations to report honestly

The official test split had already been examined during earlier experiments, so the final 95.64% is useful evidence but not a completely untouched benchmark. The real-handwriting check contains only 26 samples, and the dot rule is a narrow heuristic rather than a learned improvement to the CNN. Performance on larger, independently collected handwriting and on full words or sentences has not been established.

Current checkpoint: **emnist_vgg_benchmark_val_best.pth**

Current inference steps: paper/ink preprocessing (Method C) → VGG-style CNN prediction → conservative i/l dot check.

End of report